M E R N

MERN Stack

7-Day Learning Guide

MongoDB · Express.js · React · Node.js

A complete, hands-on roadmap to go from JavaScript fundamentals to deploying a full-stack MERN application — one day at a time.

Day 1–2	**Foundations**	JS, Node.js, Express
Day 3–4	**Data & UI**	MongoDB, React
Day 5–6	**Full Stack**	API Integration, Auth
Day 7	**Ship It**	Polish & Deploy

Day 1: JavaScript Foundations

PREREQ

Solidify the JavaScript you'll use every day across the entire MERN stack. Everything in this stack is JavaScript — this is the highest-leverage day.

ES6+ essentials

{ }
let & const
Use const by default, let when you need to reassign. Never use var — it has scoping issues.

=>
Arrow functions
const greet = (name) => `Hello ${name}`; — Short syntax, used everywhere in React.

...
Destructuring
const { name, age } = user; — Extract values from objects/arrays. React's useState uses this.

+
Spread & rest
Spread (...arr) copies/merges. Rest (...args) collects. Same syntax, opposite jobs.

``
Template literals
Backtick strings with ${expression} — cleaner than string concatenation.

Array & object methods

M
.map()
Transform each item: arr.map(x => x * 2). Returns new array. Used in React to render lists.

F
.filter()
Keep matching items: arr.filter(x => x > 5). Used for search/filter features.

R
.reduce()
Boil down to one value: arr.reduce((sum, x) => sum + x, 0). Great for totals.

?
.find()
Get first match: arr.find(x => x.id === 2). Used to select specific records.

O
Object.*
Object.keys(), .values(), .entries() — turn objects into iterable arrays.

Async JavaScript

P
Promises
An object that says 'I'll give you a value later.' States: pending, resolved, rejected.

A
async/await
The clean way to handle async: const data = await fetch(url). Always wrap in try/catch.

F
fetch API
Built-in way to make HTTP requests: fetch('/api/notes').then(res => res.json())

Modules & npm

E
ES modules
export/import syntax. Named exports use { }, default exports don't.

N

npm basics
npm init -y, npm install express, npm run dev — the package manager for Node.js.

Tip: *Spend 3–4 hours here. Practice in the browser console or a Node.js file. Try fetching data from jsonplaceholder.typicode.com.*

Day 2: Node.js & Express Basics

BACKEND

Build your first server and learn how HTTP actually works. By the end of today, you'll have a working REST API.

N **Node.js core**
Event loop, fs module, path module, process.env for configs, nodemon for auto-restart.

Ex **Express setup**
express(), app.listen(port), middleware concept, the req/res cycle.

R **REST routes**
GET, POST, PUT, DELETE endpoints. Route params (:id) and query strings (?search=).

MW **Middleware**
express.json() to parse bodies, cors for cross-origin, custom error handlers.

Your first Express server

```
const express = require('express');
const app = express();
app.use(express.json());

let notes = [];

app.get('/api/notes', (req, res) => {
  res.json(notes);
});

app.post('/api/notes', (req, res) => {
  const note = { id: Date.now(), ...req.body };
  notes.push(note);
  res.status(201).json(note);
});

app.delete('/api/notes/:id', (req, res) => {
  notes = notes.filter(n => n.id !== Number(req.params.id));
  res.json({ message: 'Deleted' });
});

app.listen(3000, () => console.log('Server on port 3000'));
```

Tip: *Build a simple API with 4 CRUD routes for 'notes'. Test every endpoint with Postman or Thunder Client in VS Code.*

Day 3: MongoDB & Mongoose

DATABASE

Store real data. Learn schemas, queries, and how documents differ from SQL tables.

DB **MongoDB basics**
Document-based (JSON-like), no rigid schema, collections instead of tables. Set up free Atlas cluster.

S **Mongoose schemas**
Define structure: types, required, default, enum, timestamps. Create models from schemas.

Q **CRUD operations**
find(), findById(), create(), findByIdAndUpdate(), findByIdAndDelete().

V **Validation**
Built-in validators (required, minlength, enum), custom validators, error handling.

Mongoose model example

```
const mongoose = require('mongoose');

const noteSchema = new mongoose.Schema({
  title:   { type: String, required: true, trim: true },
  content: { type: String, required: true },
  done:    { type: Boolean, default: false },
}, { timestamps: true });

module.exports = mongoose.model('Note', noteSchema);
```

Connecting to MongoDB

```
const mongoose = require('mongoose');
require('dotenv').config();

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('Connected to MongoDB'))
  .catch(err => console.error('Connection failed:', err));
```

Tip: *Connect your Day 2 Express API to MongoDB. Replace the in-memory array with real Mongoose queries. Your notes now persist!*

Day 4: React Fundamentals

FRONTEND

Build UIs with components, state, and props. React thinks in components — small, reusable pieces.

JSX & components
Function components return JSX (HTML-like syntax). Props pass data from parent to child.

S **useState**
const [count, setCount] = useState(0); — React re-renders when state changes.

E **useEffect**
Side effects: fetch data on mount, subscribe to events. Dependency array controls when it runs.

L **Lists & keys**
items.map(item =>). Keys help React track which items changed.

React component example

```
import { useState, useEffect } from 'react';

function NoteList() {
  const [notes, setNotes] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/notes')
      .then(res => res.json())
      .then(data => { setNotes(data); setLoading(false); })
      .catch(err => console.error(err));
  }, []); // empty array = run once on mount

  if (loading) return Loading...;

  return (

      {notes.map(note => (
        {note.title}
      ))}

  );
}
export default NoteList;
```

Tip: *Create a React app with Vite: npm create vite@latest my-app -- --template react. Build a UI with hardcoded data first, then connect to the API on Day 5.*

Day 5: Connecting Frontend to Backend

FULL STACK

The magic moment — React talks to Express which talks to MongoDB. Your app becomes real.

F **Fetch / Axios**
Make HTTP requests from React: GET to list, POST to create, DELETE to remove.

C **CORS setup**
Enable cross-origin requests in Express: app.use(cors()). Or use Vite proxy in dev.

H **Custom hooks**
useNotes() hook to encapsulate all API logic. Separates data fetching from UI.

E **Error handling**
try/catch in async calls, loading spinners, user-friendly error messages.

Custom hook pattern

```
import { useState, useEffect } from 'react';

export function useNotes() {
  const [notes, setNotes] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const fetchNotes = async () => {
    try {
      const res = await fetch('/api/notes');
      const data = await res.json();
      setNotes(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  const addNote = async (title, content) => {
    const res = await fetch('/api/notes', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title, content }),
    });
    const newNote = await res.json();
    setNotes(prev => [...prev, newNote]);
  };

  useEffect(() => { fetchNotes(); }, []);
  return { notes, loading, error, addNote };
}
```

Tip: *This is the most rewarding day. When you see data flow from your React form through Express into MongoDB and back — that's the full MERN loop.*

Day 6: Authentication & Routing

ADVANCED

Add users, login, protected routes, and multi-page navigation to your app.

JWT **JWT auth**
bcrypt to hash passwords, jsonwebtoken to create/verify tokens. Login returns a token.

P **Protected routes**
Express middleware checks the JWT token before allowing access to protected endpoints.

RR **React Router**
BrowserRouter, Routes, Route, Link, useNavigate — multi-page SPA navigation.

Ctx **Auth context**
createContext + useContext to share auth state globally. PrivateRoute component.

Auth middleware (Express)

```
const jwt = require('jsonwebtoken');

const auth = (req, res, next) => {
  const token = req.header('Authorization')?.replace('Bearer ', '');
  if (!token) return res.status(401).json({ error: 'Access denied' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.userId = decoded.userId;
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
};

// Use it:  app.get('/api/notes', auth, getNotesHandler);
```

Tip: *Each user should only see their own notes. Filter by req.userId in your database queries.*

Day 7: Polish & Deploy

SHIP IT

Make it production-ready and put it on the internet. Your goal: a live URL you can share.

UI **Polish the UI**
Responsive layout, toast notifications, form validation, clean consistent design.

ENV **Environment vars**
.env files with dotenv. Separate dev/prod configs. Never commit secrets!

D **Deploy**
Backend: Render or Railway (free tier). Frontend: Vercel or Netlify. Database: MongoDB Atlas.

N **Next steps**
TypeScript, Redux/Zustand, testing with Jest/Vitest, Next.js for full-stack React.

Deployment checklist

- Set all environment variables in your hosting platform
- Update CORS origin to your frontend's deployed URL
- Set NODE_ENV=production in your backend
- Build your React app: npm run build
- Test the deployed version end-to-end
- Add a README.md with setup instructions

***Tip:** Deploy early in the day so you have time to debug. Environment variable issues are the #1 deployment problem.*

MERN architecture at a glance

Layer	Technology	Role	Key concepts
Frontend	**React**	User interface	Components, state, hooks, JSX
API layer	**Express.js**	HTTP handling	Routes, middleware, REST
Runtime	**Node.js**	Server runtime	Event loop, npm, async I/O
Database	**MongoDB**	Data storage	Documents, Mongoose, Atlas

The data flow: React (browser) sends HTTP requests to Express (server), which queries MongoDB (database) via Mongoose, then returns JSON back through Express to React for rendering.

You've got the roadmap. Now build something!